

CHAPTER 9

Exception & File Handling

Java Exception Handling:

Exception Handling in Java is a mechanism used to handle both compile-time (checked) and runtime (unchecked) exceptions, allowing a program to continue execution smoothly even in the presence of errors.

Handles abnormal conditions that occur during program execution.

Helps maintain program stability by preventing unexpected termination.

```
class Geeks{  
    public static void main(String[] args) {  
  
        int n = 10;  
        int m = 0;
```

```

        try {
            int ans = n / m;
            System.out.println("Answer: " + ans);
        } catch (ArithmeticException e){
            System.out.println("Error: Division by
0!");
        }
    }
}

```

Finally Block:

The finally block executes after the try and catch blocks in most situations, whether an exception arised or not. It is typically used for closing resources such as database connections, open files, or network connections.

Finally may not execute in cases like:

System.exit()

JVM crash

infinite loop before finally

```
class FinallyExample {  
    public static void main(String[] args){  
  
        int[] numbers = { 1, 2, 3 };  
        try {  
            //      This      will      throw  
            ArrayIndexOutOfBoundsException  
  
            System.out.println(numbers[5]);  
  
        }  
        catch  
        (ArrayIndexOutOfBoundsException e){  
  
            System.out.println("Exception caught: " +  
e);  
  
        }  
    }  
}
```



```
        finally{
            System.out.println("This
block always executes.");
        }
        System.out.println("Program
continues...");
    }
}
```

throw and throws Keywords:

1. throw: Used to explicitly throw a single exception. We use throw when something goes wrong (or “shouldn’t happen”) and we want to stop normal flow and hand control to exception handling.

```
class Demo {
    static void checkAge(int age) {
```

```
        if (age < 18) {
            throw new
IllegalArgumentException("Age must be 18
or above");
        }
    }

    public static void main(String[] args) {

        checkAge(15);
    }
}
```

Output:

```
Exception in thread "main"
java.lang.IllegalArgumentException: Age
must be 18 or above
    at Demo.checkAge(Demo.java:5)
    at Demo.main(Demo.java:11)
```

2. throws: Declares exceptions that a method might throw, informing the caller to handle them. It is mainly used with checked exceptions (explained below). If a method calls another method that throws a checked exception, and it doesn't catch it, it must declare that exception in its throws clause.

```
import java.io.*;
```

```
class Demo {
```

```
    // Method declares that it may throw  
    IOException
```

```
    static void readFile(String fileName)  
    throws IOException {
```

```
        // Using try-with-resources to  
        automatically close FileReader
```

```
        try (FileReader file = new
```



```
FileReader(fileName)) {  
    int data;  
    while ((data = file.read()) !=  
-1) {  
        System.out.print((char)  
data); // Read and print file content  
    }  
}  
// No need for finally block to  
close the resource  
}
```

```
public static void main(String[] args) {  
  
    try {  
        readFile("test.txt"); // Attempt  
to read file  
    } catch (IOException e) {  
        System.out.println("File not  
found or error reading file: " +  
e.getMessage());  
    }  
}
```

```
        System.out.println("\nProgram
continues after file operation.");
    }
}
```

Java Multiple Catch Block:

```
public class GFG {
    public static void main(String[] args){

        try{

            // ArithmeticException
            int a = 10 / 0;
            int arr[] = new int[5];

            //
            ArrayIndexOutOfBoundsException
            arr[10] = 50;
        }
        catch (ArithmeticException e) {
```



```

        System.out.println(
            "Arithmetic Exception
occurred");
    }
    catch
(ArrayIndexOutOfBoundsException e) {
        System.out.println(
            "Array Index Out Of
Bounds Exception occurred");
    }
    catch (Exception e) {
        System.out.println(
            "General Exception
occurred");
    }
}
}

```

```

public class MultiCatchExample {
    public static void main(String[] args) {
        try {
            int a =
Integer.parseInt("ABC"); //
NumberFormatException
            int b = 10 / 0;
// ArithmeticException
        } catch (NumberFormatException
| ArithmeticException ex) {

System.out.println("Exception caught: " +
ex);

        }

    }

}.

```